

Trajectory Prediction for Autonomous Vehicles using GRIP++

Jaydev Dhanji

*A Thesis submitted in partial fulfillment
of the requirements for the degree of*

**Bachelor of Science in
Robotics and Intelligent Systems**

Submission: June 27, 2025

Supervisor: Prof.Dr. Francesco Maurelli

Statutory Declaration

Family Name, First Name	Dhanji, Jaydev
Matriculation number	30006523
Kind of thesis submitted	Bachelor Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

Date, Signature

Acknowledgements

I would like to express my sincere gratitude to my research supervisor, Prof. Dr. Francesco Maurelli, for providing immense and selfless support, advice and insight throughout the entire process. His guidance and constructive feedback have been crucial in shaping the direction of the thesis, leading it to its final form. This work would not have realized its full potential without his insightful counsel and helpful criticism. I consider myself really fortunate to have had a mentor who has been so committed to my journey.

A special mention goes to my parents, Mr. Atul Dhanji and Mrs. Sudha Dhanji. My parents have my sincere gratitude for their unwavering love, tolerance, and faith in my skills, which have given me the willpower and inspiration to face any obstacle head-on. Their selflessness and unwavering support have been a beacon of hope, inspiration and optimism. I was able to concentrate on my studies with confidence and tenacity because of their assistance.

Lastly, I would like to acknowledge the emotional support and love of my colleagues and friends. Their moral support and motivation made this journey more fulfilling. During challenging moments, their words of encouragement, empathy, and emotional support kept me balanced and driven. This scholastic trip was made much more pleasurable and rewarding by laughter, shared experiences, and thought-provoking discussions.

This thesis would not have been possible without all the invaluable contributions and constant support from these remarkable people, and I am truly grateful for their understanding.

Abstract

Safe and intelligent autonomous vehicle navigation, especially in complicated urban scenarios, depends on accurate trajectory prediction of surrounding agents. In order to predict future trajectories by capturing both spatial and temporal interactions among several individuals, this thesis explores the use of a graph neural network model called GRIP++ (Graph-based Interaction-aware Prediction). A multi-agent prediction task is used to formulate the problem, in which all traffic participants' future positions are collectively calculated using their historical trajectories. The model's performance in urban driving scenarios is assessed using the JAAD dataset, which offers actual pedestrian-vehicle interactions.

Input preprocessing, spatiotemporal graph convolution, and a Seq2Seq prediction framework compose the modular architecture used to develop GRIP++. The model is evaluated using evaluation metrics like Average Displacement Error (ADE) and Final Displacement Error (FDE) after being trained in GPU-accelerated environments using the PyTorch toolkit. According to experimental data, GRIP++ predicts socially-aware trajectories with good accuracy; however, computational limitations and agent behaviour variability affect performance. In order to generate more accurate results and context-aware predictions, further work suggests integrating transformer-based architecture into the model and implementing it in real-time in vehicles using effective graph creation techniques and model compression.

Table of Contents

1	Introduction	5
2	GRIP++ Model Description	7
2.1	Problem Formulation	7
2.1.1	Input Processing Model	8
2.1.2	Graph Convolution Model	9
2.1.3	Trajectory Prediction Model	10
3	Methodology	11
3.1	Data Selection	11
3.2	JAAD Dataset Overview	11
3.3	Data Preparation	12
3.3.1	Data Conversion	12
3.3.2	Heading Calculations	13
3.3.3	Object Type Mapping	13
3.4	Data Cleaning	13
3.5	Data Splitting	13
3.6	Training Setup	14
4	Analysis and Results	15
4.1	Further Analysis	15
4.2	Challenges and Limitations	16
4.3	Future Works	16
4.3.1	Incorporation of Transformer-Based Architectures	16
4.3.2	Real-Time Implementation and Scalability Studies	16
5	Final Conclusion	17
6	Appendix	19
6.1	Training Code	19

1 Introduction

The safe and efficient management of complex traffic circumstances continues to be a crucial research and engineering challenge as autonomous vehicles (AVs) progress from prototypes to practical implementation. **Trajectory prediction**, or the AV’s capacity to predict the future movements of dynamic agents like cars, bikes, and pedestrians, is one of the most crucial skills needed for such navigation. In order to enable safe, socially acceptable, and efficient manoeuvres, these projections are essential inputs to planning and control systems [5].

In urban settings, where agents often interact, alter their objectives, and react to intricate cues like traffic lights, crosswalks, or driving behaviours, the trajectory prediction problem is more challenging. Conventional physics-based techniques, including models of constant acceleration or velocity, are unable to generalise to such non-linear, interaction-heavy situations. Although they are more flexible, probabilistic models such as Bayesian networks and Kalman filters still have trouble with long-term forecasting and multi-agent interactions. Deep learning-based data-driven methods have become the state-of-the-art in trajectory prediction as a result of these constraints [10].

Socially-aware and interaction-aware models, which highlight how each agent’s behaviours are influenced by others in their environment in addition to their previous movements, have become more popular as a result of recent advancements in deep learning. These relational dynamics are attempted to be encoded using graph-based techniques such as **Trajectron++** [4] or **SGNet** [11], GAN-based predictors [3], and social-LSTMs [1]. By explicitly modelling the scene as a dynamic spatio-temporal graph, in which each agent is a node and the edges represent their interactions over time, **GRIP++ (Graph-based Interaction-aware Trajectory Prediction)** stands out among them [6]. The original GRIP model is further developed by GRIP++, which combines static and dynamic edges. This allows for the identification of both short-term intentions and longer-term behavioural trends. Real-time applications are a crucial requirement for AVs to operate in dense environments. This is facilitated by the model’s great computational efficiency.

Realistic and varied datasets are essential for efficiently assessing trajectory prediction models. Although the majority of current benchmarks, such as **ETH/UCY** [7] and **nuScenes** [2], are helpful for predicting generic motion, they frequently lack the detailed behavioural annotations needed to comprehend pedestrian intent. On the other hand, the **JAAD (Joint Attention for Autonomous Driving)** dataset [8] offers more than 300 video clips that include thorough annotations of pedestrian behaviours, such as head position, motion status, and environmental elements like crosswalk locations or weather. These features make JAAD an extremely helpful tool for studying pedestrian-vehicle interactions, especially in urban and suburban environments where these interactions are frequent and complex.

GRIP++’s interaction modelling framework is well-suited for adaptation to pedestrian-rich landscapes like those in JAAD, even though it was initially created for vehicle-centric situations (like **ApolloScape**). Effective use, however, necessitates careful consideration of variations in agent density, subtleties in human behaviour, and visual modality distinctions (e.g., dashboard camera viewpoints). In this thesis, we modify the GRIP++ architecture to fit the JAAD dataset and examine how well it predicts the paths of pedestrians.

In particular, we aim to respond to the following queries:

- To what extent does GRIP++ generalise from car-centric urban scenarios to pedestrian-centric ones?
- What accuracy vs. speed trade-offs exist when using GRIP++ on egocentric first-person video data?
- Can agents with unclear or context-dependent purpose have their prediction robustness increased by using joint interaction graphs?

Through both quantitative experiments and qualitative research, this work aims to demonstrate that interaction-aware trajectory forecasting models such as GRIP++, when suitably modified, can be a useful tool in enabling safer, more human-compatible autonomous navigation.

2 GRIP++ Model Description

Deep learning-based techniques, such as LSTM-based techniques, have been presented recently for trajectory prediction and manoeuvre classification; nevertheless, these models necessitate optimal road circumstances, such as clear lanes and the absence of any other kind of traffic agent. As a result, scholars started investigating possible solutions.

Later, TraPhic, a hybrid LSTM-CNN model for trajectory prediction, was released. This model uses a semi-elliptical region (horizon) in front of each road agent to describe horizon-based interactions, which implicitly simulates each road agent’s driving conduct. It also considers the different behaviours and dynamics of different road agents. Every road agent is modelled using an LSTM. The hidden states of all the agents in the designated neighbourhood are used to produce a neighbourhood map, while the hidden states of the horizon agents are combined to form a horizon map. The accuracy of such a method is limited by the computing cost and the loss of some movement information caused by CNN’s pooling of the concealed states of neighbouring agents.

In [42], Luo et al. presented a convolutional network for motion forecasting, tracking, and quick object detection. Their approach performs 3D convolutions across space and time using LiDAR data from bird’s eye views as input. They think that because the model can learn aspects of acceleration and velocity from the input of several frames, such a structure can predict motion. All objects’ visual characteristics are represented in the same feature map, though, because the predicting branch only requires the 3D convolutional feature map as input. As a result, the model becomes distracted and is unable to function well in an environment with a lot of items.

2.1 Problem Formulation

Before presenting the suggested approach, a trajectory prediction problem was defined as one that uses the trajectory histories of all objects in a scene to forecast their future locations. Consequently, the model’s inputs, X , are the trajectory histories of every item that has been observed, which are defined as follows:

$$X = [p^{(1)}, p^{(2)}, \dots, p^{(t_h)}]$$

where n is the number of items observed and $p^{(t)}$ represents the coordinates of every object detected at time t .

We claim that for an autonomous driving automobile, it makes more sense to estimate future positions for every observed object at once, since we supply the model with the track histories of every object. Therefore, the proposed model produces Y , the predicted future positions of all observed objects from time step $t_h + 1$ to $t_h + t_f$, as opposed to just predicting the position of one specific object as is done in existing models:

$$Y = [p^{(t_h+1)}, p^{(t_h+2)}, \dots, p^{(t_h+t_f)}]$$

where $p^{(t)}$ is the same as in Equation (2), and t_f is the prediction horizon.

The proposed **GRIP++** framework consists of three components:

1. **Input Preprocessing Model**
2. **Graph Convolution Model**
3. **Trajectory Prediction Model**

2.1.1 Input Processing Model

A. Input Representation:

- The raw input is transformed into a specific format to facilitate efficient computation.
- The information is represented as a 3D array F_{input} with size $(n \times t_h \times c)$, where:
 - * n is the number of objects.
 - * t_h is the number of observed time steps.
 - * c is set to 2 to represent the x and y coordinates of each object.
- Forecasting an object’s velocity is simpler than predicting its position, so the velocity is computed using:

$$v(t) = p^{(t-1)} - p^{(t)}$$

before the data is entered into the model.

B. Graph Construction:

- In GRIP++, the interaction between objects is modeled as an undirected graph $G = \{V, E\}$, with each node representing a traffic object at a particular time step.
- The node set is defined as:

$$V = \{v_{it} \mid i = 1, \dots, n, t = 1, \dots, t_h\}$$

where v_{it} represents the coordinate feature vector of the i -th object at time t .

- The edge set E consists of:
 - * **Spatial edges** (E_s): representing interactions between two objects at the same time step:

$$E_s = \{v_{it}v_{jt} \mid (i, j) \in D\}$$

where D is the collection of nearby objects.

- * **Temporal edges** (E_f): linking each object to itself in the next time step:

$$E_f = \{v_{it}v_{i(t+1)}\}$$

capturing the object’s trajectory over time.

- The graph is represented by an adjacency matrix $A = \{A_0, A_1\}$:
 - * A_0 : identity matrix for self-connections (temporal edges).
 - * A_1 : adjacency matrix for spatial connections.

Both matrices are of size $(n \times n)$, where:

$$A_0[i][j] \text{ or } A_1[i][j] = \begin{cases} 1, & \text{if edge } \langle v_{it}, v_{jt} \rangle \in E \\ 0, & \text{otherwise} \end{cases}$$

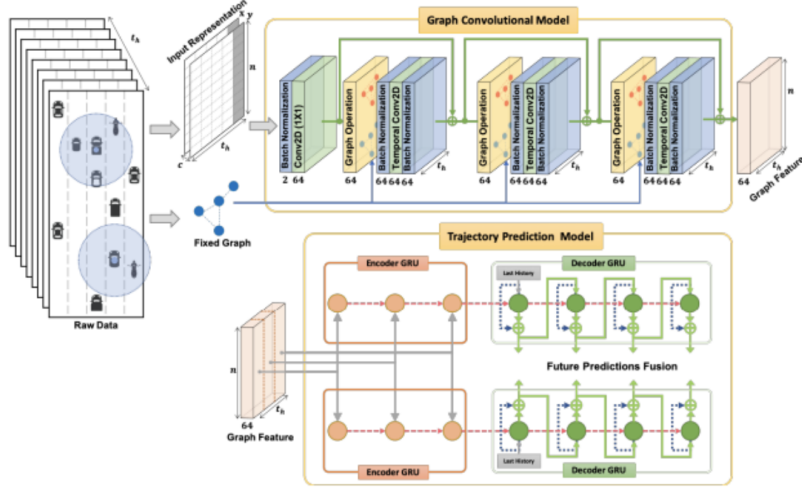


Figure 1: GRIP++ Model Scheme

2.1.2 Graph Convolution Model

This model increases the number of channels by passing a preprocessed input data $F_{\text{input}} := R^{n \times t_h \times c}$ through a 2D convolutional layer with a (1×1) kernel size, as depicted in 1 as “Conv2D (1×1) ”. In order to assist the model in learning an effective representation for the trajectory prediction task, it is intended to transfer the two-dimensional input data (x and y coordinates) into a higher-dimensional space. As a result, the output has the form $(n \times t_h \times C)$, where C is the new channel count. C is 64 in 1.

Afterwards, the input data is passed into a number of graph operations and temporal convolutions. Graph operations deal with the interaction between objects in spatial space, while temporal convolutions record valuable temporal information like an object’s motion pattern. In order to analyse the input data both spatially and temporally in each alternate layer, one Temporal Convolution layer is added at the end of each Graph Operation layer, as seen in 1, which also shows three Graph Operation layers and three Temporal Convolution layers. Additionally, batch normalization layers are included to increase the model’s stability during training. The model’s ability to transfer greater gradients to earlier layers — which may learn as quickly as the end layers — is verified via skip connections, which are represented by green polylines.

A. Graph Operation Layer: A graph operation layer considers how items interact with one another. In each Graph Operation layer, there are two graphs:

1. a Fixed Graph, which is the adjacency matrix A in 1 and the blue graph symbol above, built using the current input; and
2. a trainable graph, G_{train} , which is the orange graph symbol in the Graph Operation Model block in 1.

To ensure that the value range of feature maps remains unchanged following graph operations, the Fixed Graph A is normalised using the following equation:

$$\Lambda_j^{-\frac{1}{2}} A_j \Lambda_j^{-\frac{1}{2}} = G_j^{\text{fixed}} \quad (1)$$

where $\Lambda_j^{-\frac{1}{2}}$ is computed as:

$$\Lambda_j^{ii} = \sum_k (A_j^{ik}) + \alpha, \quad (2)$$

and α is set to 0.001 to avoid empty rows in A_j .

The Fixed Graph, G_{fixed} , may not be able to accurately depict item interactions because it is built using a manually created rule. To overcome this limitation, the authors added the Fixed Graph to the trainable graph. The output feature map, f_{graph} , is therefore determined as follows:

$$f_{\text{graph}} = \sum_{j=0}^1 \left(G_{\text{fixed}}^j + G_{\text{train}}^j \right) f_{\text{conv}} \quad (3)$$

when a Graph Operation layer receives an input f_{conv} from the preceding layer. f_{graph} is of size $(n \times t_h \times C)$ since the Graph Operation layers do not alter the feature sizes.

- B. Temporal Convolutional Layer:** A Temporal Convolutional layer receives the generated feature $f_{\text{graph}} := R^{n \times t_h \times C}$. To drive data processing in the temporal dimension, a temporal layer’s kernel size is set to (1×3) . To ensure that each layer produces an output feature map of the desired size, paddings and strides are added accordingly.

2.1.3 Trajectory Prediction Model

This model is made up of many networks that are trained with various weights but have the same Seq2Seq structure. 1 shows two Seq2Seq networks. Each network takes as its input the Graph Feature, which is produced by the Graph Convolutional Model. The Graph Feature’s feature vectors are supplied into the appropriate input cell of the Encoder GRU (Gated Recurrent Unit) at each temporal dimension. The location coordinates at the current time step are then predicted by feeding the Decoder GRU with the hidden feature of the Encoder GRU and the object coordinates. The coordinates of the objects at the “Last History” phase serve as the input for the first decoding step, and the current step’s output is then passed into the subsequent GRU cell. 1 shows this as the grey “Last History” boxes that match the grey input representation column. The aforementioned decoding procedure should be carried out repeatedly until the model predicts the positions for each anticipated time step $((t_f))$ in the future. The authors have included a residual connection, represented by blue dashed lines in 1, between the input and the output of each cell of the Decoder GRU in order to anticipate the velocity change because the traffic agents’ velocity in a particular scene is not constant. The expected velocities results are averaged at each time step once the Seq2Seq networks’ projected results are obtained. To convert the projected results back to (x, y) coordinates, the predicted velocities $(\Delta x, \Delta y)$ are averaged and then added back to the last historical location $p^{(t_h)}$.

3 Methodology

3.1 Data Selection

Understanding pedestrian behaviour and purpose is essential in urban, pedestrian-rich situations, where autonomous driving trajectory prediction is particularly difficult. Conventional datasets for this job, such as ETH/UCY, nuScenes, and KITTI, provide little behavioural or contextual annotations for pedestrians and mostly concentrate on cars or structured motion in crowd situations.

3.2 JAAD Dataset Overview

To investigate pedestrian and driver behaviour in realistic environments, the Joint Attention in Autonomous Driving (JAAD) dataset [9] was created.

- It includes 346 high-definition video clips taken in metropolitan settings in North America and Europe using dashboard cameras mounted on cars.
- Detailed annotations of pedestrian boundary boxes, frame by frame.
- Crossing, standing, staring, waving, and other behavioural annotations are included.
- Contextual factors include traffic signals, location (such as a street or intersection), weather, and time of day.

The focus on Joint Attention, or simulating how drivers and pedestrians jointly affect one another’s behaviour in shared traffic settings, is what distinguishes JAAD. Subtle indicators like head direction, hesitating before crossing, or pausing in the middle of a crosswalk are examples of this.

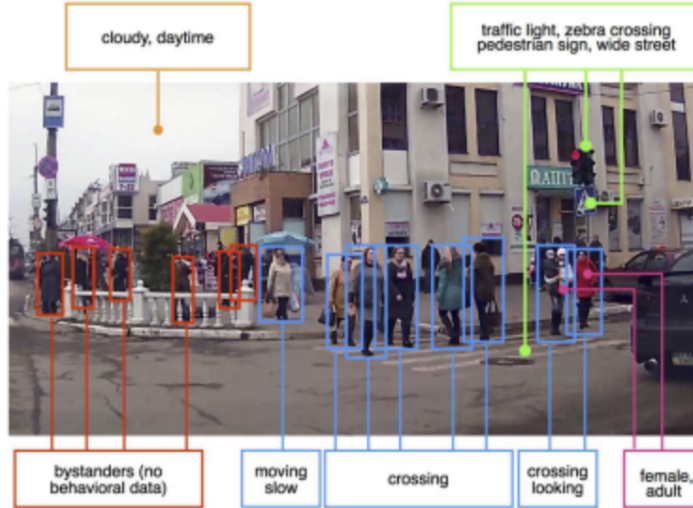


Figure 2: Sample of the dataset showing behavioural and contextual annotations

Bounding boxes, behavioural labels, gender, age of pedestrians crossing or planning to cross, and contextual labels like weather and time of day are examples of annotations included in the JAAD Dataset.

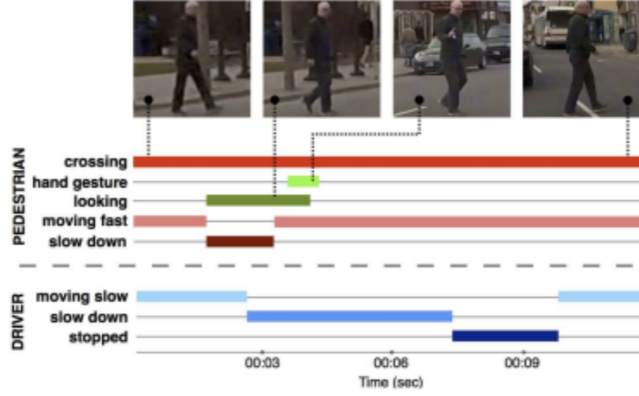


Figure 3: A video frame showing behavioural attributes of the driver and pedestrian

Time-stamped behavioural markers that illustrate the order of events during the crossing

Dataset	# ped. samples	# frames	occ. labels	temporal corr.	video sequence	behavioral data	context data	weather variation
INRIA[4]	1.8k	2.5k						
Daimler[10]	72k	28.5k			x			
Caltech[8]	347k	250k	x	x	x			
KITTI [13]	12k	80k	x	x	x			
MPD [16]	86.2k	95k	x	x	x			
Our Dataset	337k	82k	x	x	x	x	x	x

Figure 4: Comparison of JAAD Dataset with other existing large scale pedestrian datasets

3.3 Data Preparation

The JAAD dataset offers XML-formatted annotations that describe the locations, movements, and interactions of pedestrians with traffic objects. But like the Apolloscape Dataset, which the developers employed for their tests, the GRIP++ program requires data in a particular format. Data conversion was done in order to modify the JAAD dataset to meet GRIP++ requirements.

3.3.1 Data Conversion

Because XML annotations for pedestrian conduct and interactions in urban settings are included in the JAAD dataset. First, a structured text format that complies with the Apolloscape dataset format was created from these XML annotations. The following is the Apolloscape data structure for every line in the text file:

However, only eight columns were taken from the JAAD annotations; the remaining

frame_id	object_id	object_type	position_x	position_y	position_z	object_length	object_width	object_height	heading
----------	-----------	-------------	------------	------------	------------	---------------	--------------	---------------	---------

Figure 5: Apolloscape Dataset Format

columns were left out, and the *position_z* and *object_{height}* are not included in the JAAD dataset.

3.3.2 Heading Calculations

The object’s orientation or direction with regard to the x-axis is indicated by the ”heading”, which is an angular measure. The location of the object’s bounding box was used to determine the heading. In particular, the `atan2` function was used to calculate the angle of the vector generated by the bounding box’s upper left and lower right corners with respect to the x-axis. This function determines the bounding box’s angular orientation in radians by calculating the arctangent of the ratio of the two provided coordinates:

$$\text{heading} = \text{atan2}(y_{\text{br}} - y_{\text{tl}}, x_{\text{br}} - x_{\text{tl}}) \quad (4)$$

where $(x_{\text{br}}, y_{\text{br}})$ is the bottom-right corner and $(x_{\text{tl}}, y_{\text{tl}})$ is the top-left corner of the enclosing box.

These computations were required since they yielded useful directional data that the GRIP++ model could utilise to more accurately forecast the future paths of moving objects.

3.3.3 Object Type Mapping

There are five object kinds in the Apolloscape dataset: pedestrians, motorcyclists and bicyclists, small vehicles, big vehicles, and others. Each object type was assigned an ID ranging from 1 to 5, respectively. The JAAD collection lacks comprehensive information about other object categories, such vehicles and cyclists, because it concentrates more on pedestrian activity and interactions. Therefore, it was anticipated that only two object types—Pedestrians and Others—would be present in the JAAD Dataset format.

3.4 Data Cleaning

Data cleaning was carried out following the dataset’s conversion to the algorithm’s needed format. This was done in order to prevent any extraneous noise during the training and testing process and to guarantee consistency with the model’s expectations. To preserve the dataset’s consistency, any missing values or empty files were either filled in or eliminated. In addition, item types classified as ”Others” did not fall within any of the model’s established categories. This was done in order to keep the dataset complete. This procedure resulted in a reduction of the dataset from 346 files to 319 files.

3.5 Data Splitting

The `train_test_split` method from the `scikit-learn` library was employed. A particular ratio of training to testing data was recommended in accordance with the developers’ initial configuration. The data was divided based on this recommendation, with a ratio of approximately 55 to 1. This ratio was chosen to remain consistent with their methodology

and to ensure that the training data was sufficiently large while still allowing for adequate evaluation on the test data.

3.6 Training Setup

Google Colab was used for the training because it is free to use and can supply the NVIDIA GPUs needed to run the authors' original Python file, guaranteeing the processing power and CUDA support needed to train deep learning models. The GRIP++ model was implemented and trained mostly using PyTorch due to its adaptability and user-friendliness for graph-based models. Additionally, the CUDA library was utilised to guarantee GPU acceleration, which aids in accelerating backpropagation, tensor calculations, and matrix operations throughout training.

The following hyperparameters were used to train the model:

- **Learning Rate:** Based on prior tests that balanced stability and quick convergence, a value of 0.01 was selected.
- **Batch size:** In order to balance memory utilisation and the stability of the gradient updates, the batch size was set to 64 for training and 32 for validation.
- **Number of Epochs:** After experimenting with a number of different epochs, the developers discovered that 50 was a good fit. Therefore, 50 epochs of training were carried out to guarantee the model's convergence.
- **Optimiser:** Adam optimiser was employed due to its flexible learning rate capabilities, which are advantageous for models with intricate parameter spaces and architectures.

4 Analysis and Results

After the model was trained on the dataset, its performance was assessed on the test set. Predicting future trajectories based on predictions was the main objective of the evaluation; the model’s accuracy was evaluated by comparing the predictions with the ground truth.

Among the primary performance indicators were:

- **Average Displacement Error (ADE):** This calculates the average Euclidean distance over all time steps between the ground truth and the anticipated trajectories. This is significant because it shows how closely the model’s predictions match the objects’ actual movements.
- **Final Displacement Error(FDE):** This measured the difference between the anticipated and actual final positions at the most recent time step. This aids in evaluating the model’s accuracy in predicting the trajectory’s terminus, which is crucial in applications involving safety.

The table below displays the findings:

Method	ADE	FDE
TrafficPredict	7.1811	11.121
GRIP++ on ApolloScape	0.7142	1.3732
GRIP++ on JAAD	12.26	0.57

Table 1: Comparison of Different Methods for ADE and FDE

Compared to the second subset, it is clear that the model did fairly well in the first one. The model’s low FDE value indicates that it did a very good job of predicting the eventual location. However, when compared to the model on the Apolloscape dataset, the ADE falls short of expectations.

The model produced a reasonable ultimate position of the pedestrians in the second subset, however it was less accurate in predicting the paths on each time step. Since the pedestrians’ end positions are reasonably accurate when compared to GRIP++ on Apolloscape or TrafficPredict, the results are still acceptable.

4.1 Further Analysis

- **DeepLabV3+:**
 - Important aspects of the driving environment were highlighted using this deep learning model.
 - Understanding the spatial context in which the agents are moving is aided by scene segmentation.
 - The result provides a thorough understanding of how pedestrians interact with their surroundings by identifying safe zones, crosswalks, and off-limits locations.
 - As a result, the use of semantic segmentation guarantees that the predictions are both accurate and environment-contextual.

- **YOLO:**

- To monitor the pedestrians’ movements and interactions, object detection was carried out and bounding boxes were constructed around the things seen in the video frames.
- This was done in order to identify pedestrians from other agents and link the right people to their expected trajectory.
- The model’s predictions for pedestrian movements are displayed in the visualisations below.
- In order to confirm the model’s efficacy beyond quantitative measurements, this visualisation was crucial.

4.2 Challenges and Limitations

For the model to be trained effectively, high speed GPUs and CUDA-enabled environments were necessary, although these can be resource-intensive. The autonomous vehicles for which this model was created do not have such resources. Because embedded systems work within the limitations of tiny devices mounted in vehicles, algorithms with a good baseline for trajectory predictions must be compatible with them.

4.3 Future Works

4.3.1 Incorporation of Transformer-Based Architectures

Transformers are a perfect fit for improving GRIP++’s temporal modelling skills because they have lately demonstrated a great deal of promise in modelling long-range dependencies in sequence data. Transformers, as opposed to recurrent or convolutional models, can use self-attention to gather global context across all agents and time steps, potentially improving trajectory prediction in intricate multi-agent settings. Transformer-based modules can be used in place of or in addition to GRIP++’s graph neural networks in the future to create a hybrid architecture that better captures temporal dynamics and spatial relationships. Predictions may become more precise and contextually aware as a result, particularly in situations when agent behaviours are unclear or non-linear.

4.3.2 Real-Time Implementation and Scalability Studies

Despite achieving high prediction performance, GRIP++’s computational complexity makes it difficult to use in autonomous vehicles in real-time. The graph structure expands quickly as the number of agents rises, resulting in a large processing and memory overhead during attention computations and message passing. Future studies should focus on enhancing the model’s accuracy in low-latency environments by implementing more efficient graph generation methods or model compression techniques like quantisation and pruning.

5 Final Conclusion

The trajectory prediction problem for autonomous vehicles in complex urban scenarios was the focus of the thesis. Through a graph-based architecture, spatial and temporal interactions between dynamic agents were captured using the GRIP++ model. The main benchmark was the JAAD dataset, which offered actual driving situations with annotated vehicle and pedestrian behaviour. The goal of the task was to forecast future paths for every agent that was seen at the same time, taking into account the functional needs of autonomous driving systems.

Graph convolutional networks and sequential models were successfully combined in the GRIP++ architecture to represent motion dynamics and inter-agent relationships. Although there are still limitations in situations involving irregular motion, sparse data, or dynamic context changes, the model’s capacity to predict future positions with an acceptable degree of accuracy was shown by the results. The GRU-based decoder’s use of spatial-temporal graphs and residual connections helped to capture velocity trends and lower cumulative error over the prediction horizon.

An extended investigation was carried out to integrate DeepLabV3+ for semantic segmentation and YOLOv5 for object detection to increase predicted trajectory accuracy and environmental knowledge. The reliability of autonomous vehicle navigation in changing traffic situations was significantly increased by these developments, which also improved contextual awareness and enriched scene representation.

References

- [1] Alexandre Alahi et al. “Social LSTM: Human trajectory prediction in crowded spaces”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 2016, pp. 961–971.
- [2] Holger Caesar et al. “nuScenes: A Multimodal Dataset for Autonomous Driving”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.
- [3] Agrim Gupta et al. “Social GAN: Socially acceptable trajectories with generative adversarial networks”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 2018, pp. 2255–2264.
- [4] Brian Ivanovic and Marco Pavone. “The Trajectron: Probabilistic Multi-Agent Trajectory Modeling with Dynamic Spatiotemporal Graphs”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019.
- [5] Sébastien Lefèvre, David Vasquez, and Christian Laugier. “A survey on motion prediction and risk assessment for intelligent vehicles”. In: *Robotics and Autonomous Systems* 58.10 (2014), pp. 1159–1177.
- [6] Yichuan Li et al. *GRIP and GRIP++: Graph-based Interaction-aware Trajectory Prediction*. ApolloScape and arXiv. <https://arxiv.org/abs/1909.07584>. 2019–2020.
- [7] Stefan Pellegrini et al. “You’ll Never Walk Alone: Modeling Social Behavior for Multi-Target Tracking”. In: *Proceedings of the IEEE 12th International Conference on Computer Vision (ICCV)*. 2009.
- [8] Amir Rasouli, Iuliia Kotseruba, and John K Tsotsos. “Agreeing to cross: How drivers and pedestrians communicate”. In: *IEEE Intelligent Vehicles Symposium (IV)*. 2017, pp. 264–269.
- [9] Amir Rasouli, Iuliia Kotseruba, and John K Tsotsos. “Are they going to cross? A benchmark dataset and baseline for pedestrian crosswalk behavior”. In: *Proceedings of the IEEE International Conference on Computer Vision Workshops*. 2017, pp. 206–213.
- [10] Andrey Rudenko et al. “Human motion trajectory prediction: A survey”. In: *The International Journal of Robotics Research* 39.8 (2020), pp. 895–935.
- [11] Zuxuan Song et al. “SGNet: Stepwise Goal-Directed Network for Trajectory Prediction”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.

6 Appendix

6.1 Training Code

```
1 # Cell 1
2 !nvidia-smi
3 #Upload the GRIP++ model and dataset on google drive and then to
  colab
4 from google.colab import drive
5 drive.mount('/content/drive')
6
7 import torch
8 torch.cuda.is_available()
9
10 !python3 /content/drive/MyDrive/GRIP-master/data_process.py
11
12 !python3 /content/drive/MyDrive/GRIP-master/main.py
13
14 !python3 /content/drive/MyDrive/GRIP-master/visualize.py
```